

AutoGrade — Hệ thống chấm điểm code tự động

Đặc tả Use-Case (Use-Case Specification)

Đồ án môn Công nghệ phần mềm cho AI

Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM (HCMUS)

Mục lục (Table of Contents)

1. Use-case Model.....	3
1.1 Danh sách actor.....	3
1.2 Phân nhóm use-case theo nguồn gốc.....	3
1.3 Sơ đồ use-case.....	4
2. Use-case Specifications.....	4
2.1 Use-case: Đăng ký tài khoản.....	4
2.2 Use-case: Xác nhận email.....	5
2.3 Use-case: Đăng nhập.....	6
2.4 Use-case: Quên và đặt lại mật khẩu.....	7
2.5 Use-case: Tạo lớp học.....	8
2.6 Use-case: Tham gia lớp học.....	8
2.7 Use-case: Tạo bài tập.....	9
2.8 Use-case: Nộp bài tập.....	10
2.9 Use-case: Chấm điểm tự động bài tập.....	11
2.10 Use-case: Chỉnh sửa điểm bài nộp.....	12
2.11 Use-case: Công bố điểm bài tập.....	12
2.12 Use-case: Tạo bài tập tự luyện.....	13
2.13 Use-case: Làm và nộp bài tự luyện.....	14
2.14 Use-case: Xuất bảng điểm ra Excel.....	15
3. Đặc tả rút gọn các use-case <<include>>/<<extend>>.....	17

1. Use-case Model

Mô hình use case của AutoGrade gồm 2 actor người dùng (Sinh viên, Giảng viên — kế thừa từ actor tổng quát Người dùng) và 2 actor hệ thống ngoài là dịch vụ AI (Gemini API, Groq API). Toàn bộ use case được chia thành 3 nhóm và tô màu để dễ phân biệt: (1) đặc trưng hệ thống — xác thực; (2) nghiệp vụ; (3) quản trị hệ thống — quản trị dữ liệu. Các quan hệ «include» và «extend» được vẽ tường minh để phản ánh đúng luồng xử lý trong mã nguồn.

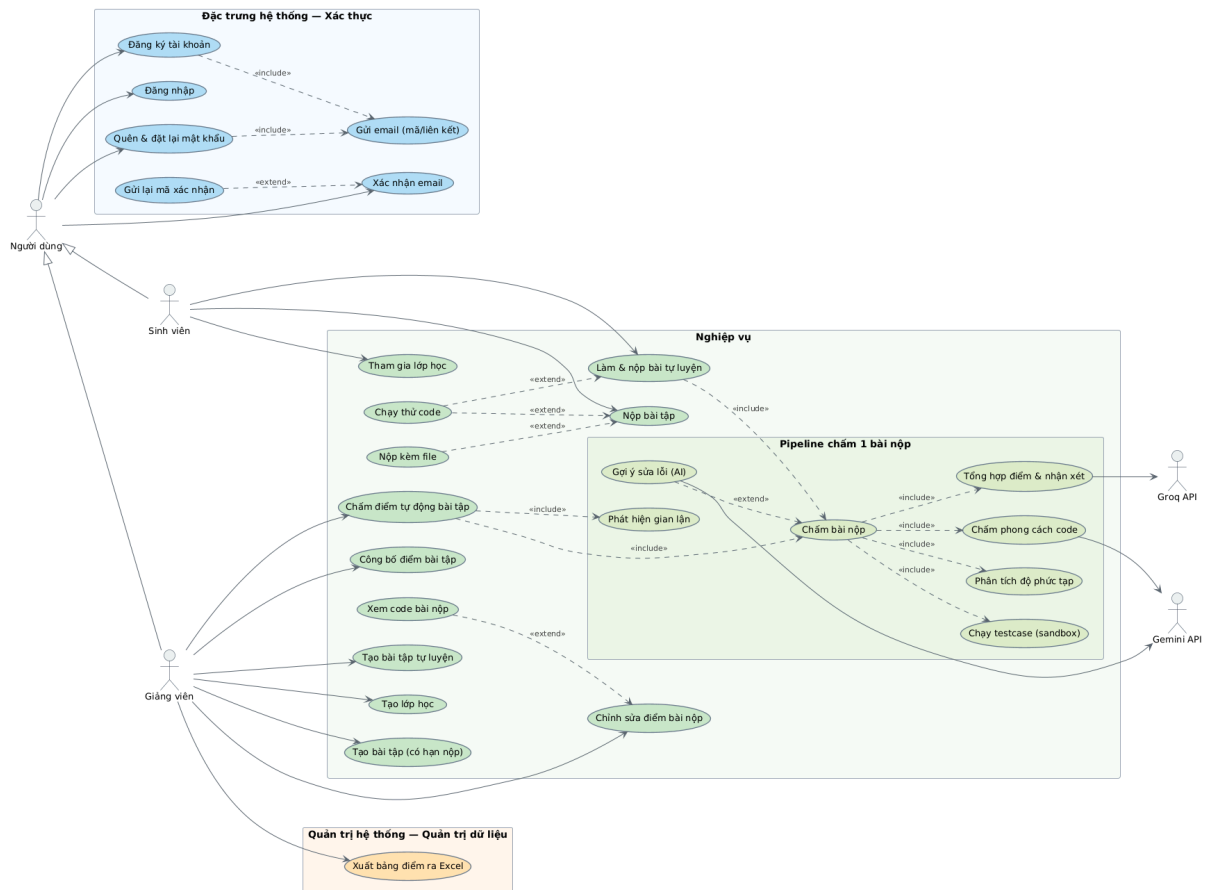
1.1 Danh sách actor

Actor	Vai trò / trách nhiệm
Người dùng (abstract)	Actor tổng quát, cha của Sinh viên và Giảng viên; sở hữu các use case xác thực dùng chung.
Sinh viên	Tham gia lớp, nộp bài, làm bài tự luyện, xem điểm.
Giảng viên	Tạo lớp, ra đề, chấm tự động, chỉnh điểm, công bố điểm, xuất báo cáo.
Gemini API (ngoài)	Dịch vụ AI: chấm phong cách code và gợi ý sửa lỗi (chế độ debug).
Groq API (ngoài)	Dịch vụ AI: viết nhận xét tổng hợp khi tổng kết điểm.

1.2 Phân nhóm use case theo nguồn gốc (mã màu)

Màu	Use case trong nhóm	Nguồn gốc use case
Đặc trưng hệ thống — Xác thực	Đăng ký tài khoản, Xác nhận email, Đăng nhập, Quên & đặt lại mật khẩu, Gửi email, Gửi lại mã xác nhận	Phát sinh từ ĐẶC TRƯNG HỆ THỐNG (một phần mềm nào cũng cần): xác thực, quản lý phiên, bảo mật tài khoản.
Nghiệp vụ	Tạo/Tham gia lớp học, Tạo bài tập, Nộp bài, Chấm điểm tự động, Chỉnh điểm, Công bố điểm, Tạo & làm bài tự luyện, cùng các UC con của pipeline chấm điểm	Phát sinh từ NGHIỆP VỤ cốt lõi: quản lý lớp học, ra đề – nộp bài – chấm điểm tự động.
Quản trị hệ thống — Quản trị dữ liệu	Xuất bảng điểm ra Excel	Phát sinh từ QUẢN TRỊ HỆ THỐNG, cụ thể là QUẢN TRỊ DỮ LIỆU (kết xuất báo cáo). Không có “quản trị người dùng” vì chức năng đó đã do use case Đăng ký đảm nhiệm.

1.3 Sơ đồ use case



Hình 1. Sơ đồ use case AutoGrade

2. Use-case Specifications

Đặc tả chi tiết 14 use case chính theo mẫu RUP. Mỗi use case được gắn nhãn nhóm màu tương ứng ở Mục 1.2.

2.1 Use-case: Đăng ký tài khoản

Nhóm: Đặc trưng hệ thống — Xác thực

Use case Name	Đăng ký tài khoản
Brief description	Use case mô tả cách người dùng (Sinh viên hoặc Giảng viên) tạo tài khoản mới trên AutoGrade bằng email trường và mật khẩu.
Actors	Sinh viên, Giảng viên
Basic Flow	- Người dùng vào trang “Đăng ký”, chọn vai trò “Sinh viên” hoặc “Giảng viên”. - Người dùng nhập Họ và tên; nếu vai trò là Sinh viên thì nhập thêm MSSV. - Người dùng nhập Email trường và Mật khẩu (tối thiểu 6 ký tự). - Người dùng nhấn nút “Đăng ký”.

	5. Hệ thống kiểm tra email chưa được đăng ký (đã xác nhận), mã hoá mật khẩu bằng bcrypt, tạo bản ghi người dùng với email_verified = false và sinh mã xác nhận 6 số. 6. Hệ thống gửi email chứa mã xác nhận (ở môi trường DEV, mã được in ra console thay vì gửi thật) và hiển thị màn hình “Xác nhận email của bạn”. 7. Use case kết thúc; luồng nghiệp vụ tiếp tục ở use case “Xác nhận email”.
Alternative Flows	Alternative flow 1: Email đã được đăng ký và đã xác nhận 1. Từ bước #5 của luồng cơ bản, hệ thống phát hiện email đã tồn tại và email_verified = true. 2. Hệ thống trả lỗi “Email đã được đăng ký” (HTTP 409); người dùng được yêu cầu đăng nhập hoặc dùng email khác. Alternative flow 2: Email đã đăng ký nhưng chưa xác nhận (đăng ký đang dở) 1. Từ bước #5, hệ thống phát hiện email tồn tại nhưng email_verified = false. 2. Hệ thống ghi đề họ tên/mật khẩu/vai trò/MSSV và sinh mã xác nhận mới, tiếp tục bước #6. Alternative flow 3: Dữ liệu không hợp lệ 1. Người dùng bỏ trống họ tên/email/mật khẩu, hoặc mật khẩu dưới 6 ký tự. 2. Hệ thống báo lỗi tương ứng và không tạo tài khoản.
Pre-conditions	Người dùng chưa có tài khoản đã xác nhận với email này; người dùng có quyền truy cập email đã nhập.
Post-conditions	Tài khoản được tạo ở trạng thái chưa xác nhận (email_verified = false); mã xác nhận 6 số đã được gửi tới email.
Quan hệ include / extend	«include» Gửi email (mã/liên kết): luôn gửi email chứa mã xác nhận ở cuối luồng cơ bản.

2.2 Use-case: Xác nhận email

Nhóm: Đặc trưng hệ thống — Xác thực

Use case Name	Xác nhận email
Brief description	Use case mô tả cách người dùng xác nhận email bằng mã 6 số sau khi đăng ký, để kích hoạt tài khoản.
Actors	Sinh viên, Giảng viên
Basic Flow	1. Từ màn hình sau đăng ký (hoặc link trong email), người dùng vào trang “Xác nhận email” với email đã điền sẵn. 2. Người dùng nhập mã xác nhận 6 số nhận được qua email. 3. Hệ thống kiểm tra mã nhập vào khớp với verify_token đã lưu cho email đó.

	4. Hệ thống đặt email_verified = true và xoá verify_token. 5. Hệ thống hiển thị “Xác nhận email thành công. Bạn có thể đăng nhập.” và chuyển tới trang đăng nhập.
Alternative Flows	Alternative flow 1: Mã xác nhận không đúng - Từ bước #3, mã nhập vào không khớp verify_token. - Hệ thống báo lỗi “Mã xác nhận không đúng”; người dùng có thể nhập lại hoặc bấm “Gửi lại mã”. Alternative flow 2: Gửi lại mã xác nhận «extend» - Người dùng bấm “Không nhận được? Gửi lại mã”. - Nếu email tồn tại và chưa xác nhận, hệ thống sinh mã mới và gửi lại; nếu không, hệ thống vẫn trả một thông báo chung để chống dò email. Alternative flow 3: Email đã được xác nhận từ trước - Từ bước #3, hệ thống phát hiện email_verified đã là true. - Hệ thống trả thông báo thành công như bình thường (API có tính idempotent), không báo lỗi.
Pre-conditions	Người dùng đã đăng ký tài khoản và nhận được mã xác nhận qua email.
Post-conditions	Tài khoản chuyển sang trạng thái email_verified = true và có thể đăng nhập.
Quan hệ include / extend	«extend» Gửi lại mã xác nhận: khi người dùng chủ động yêu cầu gửi lại (Alt flow 2).

2.3 Use-case: Đăng nhập

Nhóm: Đặc trưng hệ thống — Xác thực

Use case Name	Đăng nhập
Brief description	Use case mô tả cách người dùng đăng nhập bằng email và mật khẩu để nhận token truy cập các chức năng theo vai trò.
Actors	Sinh viên, Giảng viên
Basic Flow	- Người dùng vào trang “Đăng nhập”, nhập Email và Mật khẩu. - Người dùng nhấn nút “Đăng nhập”. - Hệ thống tìm người dùng theo email và so khớp mật khẩu (bcrypt.compare). - Hệ thống kiểm tra email_verified = true. - Hệ thống ký JWT (chứa id và role) với thời hạn mặc định 7 ngày và trả về cùng thông tin người dùng. - Giao diện lưu token và chuyển tới Dashboard tương ứng vai trò (Sinh viên hoặc Giảng viên).
Alternative Flows	Alternative flow 1: Email hoặc mật khẩu không đúng Từ bước #3, không tìm thấy người dùng hoặc mật khẩu không khớp.

	2. Hệ thống trả lỗi 401 “Email hoặc mật khẩu không đúng”; người dùng nhập lại. Alternative flow 2: Email chưa xác nhận 1. Từ bước #4, email_verified = false. 2. Hệ thống trả lỗi 403 “Email chưa được xác nhận”; người dùng được hướng tới use case “Xác nhận email”.
Pre-conditions	Người dùng đã có tài khoản với email đã xác nhận.
Post-conditions	Người dùng được xác thực, nhận token hợp lệ và có quyền truy cập API theo vai trò trong thời hạn của token.
Quan hệ include / extend	Không có quan hệ include/extend. Là cổng xác thực cho toàn bộ use case nghiệp vụ (đóng vai trò tiền điều kiện).

2.4 Use-case: Quên và đặt lại mật khẩu

Nhóm: Đặc trưng hệ thống — Xác thực

Use case Name	Quên và đặt lại mật khẩu
Brief description	Use case mô tả cách người dùng lấy lại quyền truy cập tài khoản khi quên mật khẩu, thông qua mã đặt lại gửi qua email.
Actors	Sinh viên, Giảng viên
Basic Flow	1. Ở trang “Quên mật khẩu”, người dùng nhập email. 2. Hệ thống tạo mã đặt lại (reset_token) hết hạn sau 1 giờ và gửi link/mã qua email nếu email tồn tại; API luôn trả một thông báo chung để chống dò email. 3. Người dùng mở link “/dat-lai-mat-khau?token=...” (hoặc nhập mã), nhập mật khẩu mới (tối thiểu 6 ký tự). 4. Hệ thống kiểm tra mã khớp reset_token và chưa hết hạn. 5. Hệ thống mã hoá và lưu mật khẩu mới, xoá reset_token và reset_expires. 6. Hệ thống hiển thị “Đặt lại mật khẩu thành công. Bạn có thể đăng nhập.” và chuyển tới trang đăng nhập.
Alternative Flows	Alternative flow 1: Mã đặt lại không đúng hoặc đã hết hạn 1. Từ bước #4, mã không khớp hoặc reset_expires đã qua thời điểm hiện tại. 2. Hệ thống báo lỗi tương ứng; người dùng phải yêu cầu lại mã mới từ đầu. Alternative flow 2: Mật khẩu mới không hợp lệ 1. Từ bước #3, mật khẩu mới dưới 6 ký tự. 2. Hệ thống báo lỗi “Mật khẩu tối thiểu 6 ký tự”.
Pre-conditions	Người dùng đã có tài khoản trên hệ thống.
Post-conditions	Mật khẩu của tài khoản được cập nhật; mã đặt lại cũ không còn hiệu lực.

Quan hệ include / extend	«include» Gửi email (mã/liên kết): luôn gửi email chứa link/mã đặt lại ở bước #2.
---------------------------------	---

2.5 Use-case: Tạo lớp học

Nhóm: Nghiệp vụ

Use case Name	Tạo lớp học
Brief description	Use case mô tả cách Giảng viên tạo một lớp học mới để quản lý bài tập và sinh viên.
Actors	Giảng viên
Basic Flow	1. Giảng viên vào Dashboard, nhấn “Tạo lớp mới”. 2. Giảng viên nhập Tên môn học, chọn Năm học và Học kỳ, nhập Mô tả (tuỳ chọn) và chọn ngày bắt đầu / kết thúc. 3. Giảng viên nhấn “Tạo lớp”. 4. Hệ thống tự sinh mã lớp (join_code) duy nhất từ chữ đầu tên môn học kèm mã ngẫu nhiên (ví dụ “CTDL-7K2P”). 5. Hệ thống lưu lớp học với Giảng viên hiện tại là chủ lớp và hiển thị lớp mới trong danh sách lớp của Giảng viên.
Alternative Flows	Alternative flow 1: Thiếu thông tin bắt buộc 1. Từ bước #3, thiếu Tên môn học, Năm học, Học kỳ, ngày bắt đầu hoặc ngày kết thúc. 2. Hệ thống báo lỗi “Thiếu thông tin lớp” và không tạo lớp.
Pre-conditions	Giảng viên đã đăng nhập.
Post-conditions	Lớp học mới được tạo với một mã lớp (join_code) duy nhất để Sinh viên dùng để tham gia.
Quan hệ include / extend	Không có quan hệ include/extend.

2.6 Use-case: Tham gia lớp học

Nhóm: Nghiệp vụ

Use case Name	Tham gia lớp học
Brief description	Use case mô tả cách Sinh viên tham gia một lớp học bằng mã lớp do Giảng viên cung cấp.
Actors	Sinh viên
Basic Flow	1. Sinh viên vào Dashboard, nhấn “Vào lớp học”. 2. Sinh viên nhập Mã lớp do Giảng viên cung cấp.

	3. Sinh viên nhấn “Vào lớp”. 4. Hệ thống tìm lớp theo join_code; nếu tồn tại, tạo bản ghi enrollment gắn Sinh viên với lớp. 5. Hệ thống hiển thị lớp học vừa tham gia trong danh sách lớp của Sinh viên.
Alternative Flows	Alternative flow 1: Mã lớp không tồn tại 1. Từ bước #4, không tìm thấy lớp với mã nhập vào. 2. Hệ thống báo lỗi “Mã lớp không tồn tại”; Sinh viên nhập lại. Alternative flow 2: Sinh viên đã ở trong lớp 1. Từ bước #4, bản ghi enrollment đã tồn tại từ trước. 2. Hệ thống bỏ qua việc tạo bản ghi trùng và vẫn hiển thị lớp như bước #5.
Pre-conditions	Sinh viên đã đăng nhập và có mã lớp hợp lệ do Giảng viên cung cấp.
Post-conditions	Sinh viên trở thành thành viên của lớp học và thấy được bài tập/tài liệu của lớp.
Quan hệ include / extend	Không có quan hệ include/extend.

2.7 Use-case: Tạo bài tập (có hạn nộp)

Nhóm: Nghiệp vụ

Use case Name	Tạo bài tập (có hạn nộp)
Brief description	Use case mô tả cách Giảng viên ra một bài tập có hạn nộp cho lớp học, kèm rubric chấm điểm và testcase.
Actors	Giảng viên
Basic Flow	1. Giảng viên vào trang chi tiết lớp học, chọn “Tạo bài tập”. 2. Giảng viên nhập Tiêu đề, chọn nguồn đề (Văn bản hoặc tải PDF), nhập Mô tả/đính kèm PDF, chọn “mục” gom bài và chọn Hạn nộp. 3. Giảng viên cấu hình Rubric: trọng số Testcase (mặc định 80%); bật/tắt và trọng số Độ phức tạp (mặc định 20%) cùng các mốc Big-O tương ứng phần trăm; bật/tắt và Ngưỡng gian lận (mặc định 85%). 4. Giảng viên thêm các testcase (input/expected output), có thể đánh dấu testcase ẩn để Sinh viên không thấy. 5. Giảng viên nhấn “Tạo bài tập”. 6. Hệ thống kiểm tra Giảng viên là chủ lớp, lưu bài tập cùng rubric và danh sách testcase. 7. Bài tập xuất hiện trong danh sách bài tập của lớp và hiển thị hạn nộp cho Sinh viên trong lớp.
Alternative Flows	Alternative flow 1: Thiếu thông tin bắt buộc 1. Từ bước #6, thiếu lớp học, Tiêu đề, Hạn nộp hoặc Rubric.

	2. Hệ thống báo lỗi “Thiếu thông tin bài tập”. Alternative flow 2: Không có quyền với lớp 1. Từ bước #6, Giảng viên hiện tại không phải chủ lớp. 2. Hệ thống trả lỗi 403 “Không có quyền với lớp này”. Alternative flow 3: Chỉnh sửa bài tập đã tạo 1. Giảng viên vào tab “Sửa đề” của bài tập, thay đổi tiêu đề/mô tả/hạn/rubric/testcase. 2. Hệ thống cập nhật bài tập và thay toàn bộ danh sách testcase tương ứng.
Pre-conditions	Giảng viên đã đăng nhập và là chủ của lớp học.
Post-conditions	Bài tập mới (kèm rubric và testcase) được tạo trong lớp, sẵn sàng để Sinh viên nộp bài.
Quan hệ include / extend	Không có quan hệ include/extend.

2.8 Use-case: Nộp bài tập

Nhóm: Nghiệp vụ

Use case Name	Nộp bài tập
Brief description	Use case mô tả cách Sinh viên nộp bài giải cho một bài tập của lớp học trước hạn nộp.
Actors	Sinh viên
Basic Flow	1. Sinh viên mở bài tập trong lớp đã tham gia, xem đề bài, hạn nộp còn lại và testcase công khai. 2. Sinh viên viết/dán code vào ô soạn code hoặc tải file lên. 3. Sinh viên nhấn “Chạy thử” để kiểm tra code với các testcase công khai (không lưu, không tính điểm). 4. Sinh viên nhấn “Nộp bài” (hoặc “Nộp lại” nếu đã nộp trước đó). 5. Hệ thống kiểm tra chưa quá hạn nộp, lưu (hoặc ghi đè) bài nộp theo cặp bài tập/sinh viên với trạng thái “submitted”. 6. Hệ thống hiển thị xác nhận đã nộp bài thành công.
Alternative Flows	Alternative flow 1: Đã quá hạn nộp 1. Từ bước #5, thời điểm hiện tại đã sau hạn nộp của bài tập. 2. Hệ thống trả lỗi 403 “Đã quá hạn nộp bài”; nút “Nộp bài” bị khoá trên giao diện. Alternative flow 2: Nộp lại trước hạn 1. Từ bước #4, Sinh viên đã có bài nộp trước đó và hạn nộp còn hiệu lực. 2. Hệ thống ghi đè bài nộp cũ (code, file, thời điểm nộp) và đặt lại các trường điểm/chấm về rỗng, trạng thái “submitted”. Alternative flow 3: Nộp kèm file «extend»

	1. Từ bước #2, Sinh viên chọn tải file code lên thay vì dán trực tiếp. 2. Hệ thống lưu file vào thư mục upload và ghi đường dẫn file vào bài nộp.
Pre-conditions	Sinh viên đã tham gia lớp học chứa bài tập; còn trong hạn nộp.
Post-conditions	Bài nộp của Sinh viên được lưu ở trạng thái “submitted”, chờ chấm.
Quan hệ include / extend	«extend» Chạy thử code (tuỳ chọn, trước khi nộp) và «extend» Nộp kèm file (khi tải file thay vì dán code).

2.9 Use-case: Chấm điểm tự động bài tập

Nhóm: Nghiệp vụ

Use case Name	Chấm điểm tự động bài tập
Brief description	Use case mô tả cách Giảng viên kích hoạt chấm điểm tự động cho toàn bộ bài nộp của một bài tập, qua pipeline 5 mô-đun.
Actors	Giảng viên (chính); Gemini API, Groq API (phụ — dịch vụ AI bên ngoài)
Basic Flow	1. Sau hạn nộp, Giảng viên mở tab “Xem & chấm điểm” của bài tập, nhấn “Chạy chấm tự động”. 2. Hệ thống chấm Gian lận trước: so khớp k-gram/winnowing giữa các bài nộp; bài có % giống cao nhất vượt ngưỡng rubric bị cho 0 điểm và đánh dấu “Gian lận”, dừng pipeline cho bài đó. 3. Với bài không gian lận, hệ thống chạy Testcase trong sandbox (biên dịch/chạy, so output đã chuẩn hoá). 4. Song song, hệ thống chạy Độ phức tạp (phân tích tĩnh cấu trúc code, map theo các mốc Big-O trong rubric) và Phong cách code (gọi Gemini API nhận xét, không tính điểm). 5. Hệ thống Tổng kết: tính điểm cuối tất định theo trọng số rubric và gọi Groq API viết nhận xét tổng hợp. 6. Hệ thống lưu điểm, % testcase, % độ phức tạp, lớp độ phức tạp, % giống bài, cờ gian lận và nhận xét AI vào từng bài nộp, chuyển trạng thái “graded”. 7. Giảng viên xem bảng điểm AI và nhận xét AI cho từng Sinh viên.
Alternative Flows	Alternative flow 1: Bài nộp không chạy được → Gợi ý sửa lỗi «extend» 1. Từ bước #3, bài nộp không biên dịch được hoặc mọi testcase đều lỗi/timeout. 2. Hệ thống rẽ sang Gemini ở chế độ tìm lỗi (debug) để chỉ ra lỗi và hướng dẫn sửa, không tính điểm tổng kết cho bài đó. Alternative flow 2: Chưa cấu hình API key Gemini/Groq 1. Từ bước #4 hoặc #5, biến môi trường khoá API để trống hoặc dịch vụ AI lỗi. 2. Hệ thống dùng nhận xét dự phòng ghép từ số liệu tất định; điểm số vẫn được tính bình thường vì không phụ thuộc kết quả AI.

	Alternative flow 3: Điểm đã công bố (khóa) 1. Từ bước #1, bài tập đã ở trạng thái “Đã công bố”. 2. Nút “Chạy chấm tự động” bị khóa, Giảng viên không thể chấm lại.
Pre-conditions	Bài tập có ít nhất một bài nộp; điểm chưa được công bố.
Post-conditions	Mỗi bài nộp có điểm AI, thống kê chấm chi tiết và nhận xét, ở trạng thái “graded”, sẵn sàng cho Giảng viên xem lại/chỉnh trước khi công bố.
Quan hệ include / extend	«include» Phát hiện gian lận (GATE, chạy trước) và «include» Chấm bài nộp (áp cho từng bài không gian lận). «extend» Gợi ý sửa lỗi (AI) khi code không chạy được.

2.10 Use-case: Chỉnh sửa điểm bài nộp

Nhóm: Nghiệp vụ

Use case Name	Chỉnh sửa điểm bài nộp
Brief description	Use case mô tả cách Giảng viên ghi đề điểm/nhận xét do hệ thống chấm tự động, dựa trên đánh giá thủ công.
Actors	Giảng viên
Basic Flow	1. Giảng viên xem bảng điểm AI của bài tập ở tab “Xem & chấm điểm”. 2. Giảng viên sửa trực tiếp ô Điểm và/hoặc ô Nhận xét cho một bài nộp. 3. Khi Giảng viên rời khỏi ô nhập, hệ thống gửi yêu cầu cập nhật điểm. 4. Hệ thống cập nhật điểm/nhận xét, đánh dấu bài nộp là “đã được Giảng viên can thiệp” (lecturer_overridden) và ghi thời điểm chấm. 5. Giao diện cập nhật lại giá trị điểm/nhận xét mới cho bài nộp đó.
Alternative Flows	Alternative flow 1: Điểm đã công bố (khóa) 1. Từ bước #2, bài tập đã ở trạng thái công bố. 2. Các ô Điểm/Nhận xét bị khóa, Giảng viên không thể sửa. Alternative flow 2: Xem chi tiết code trước khi chỉnh điểm «extend» 1. Từ bước #1, Giảng viên nhấn “Xem” trên một bài nộp để mở code đầy đủ của Sinh viên. 2. Giảng viên quay lại bảng điểm và tiếp tục từ bước #2.
Pre-conditions	Bài tập đã được chấm tự động (hoặc có bài nộp cần chấm tay); điểm chưa công bố.
Post-conditions	Điểm/nhận xét của bài nộp được cập nhật theo đánh giá của Giảng viên, được đánh dấu là đã can thiệp thủ công.
Quan hệ include / extend	«extend» Xem code bài nộp (tuỳ chọn, trước khi chỉnh điểm).

2.11 Use-case: Công bố điểm bài tập

Nhóm: Nghiệp vụ

Use case Name	Công bố điểm bài tập
Brief description	Use case mô tả cách Giảng viên hoàn tất và công bố điểm của một bài tập cho toàn bộ Sinh viên trong lớp.
Actors	Giảng viên
Basic Flow	1. Sau khi rà soát và chỉnh điểm (nếu cần), Giảng viên nhấn “Hoàn thành & công bố”. 2. Hệ thống hiển thị hộp thoại xác nhận vì hành động không thể hoàn tác. 3. Giảng viên xác nhận. 4. Hệ thống gán 0 điểm cho các Sinh viên trong lớp chưa nộp bài, kèm nhận xét “Không nộp bài trước hạn — 0 điểm.”. 5. Hệ thống đặt bài tập ở trạng thái đã công bố và đánh dấu toàn bộ bài nộp liên quan là công khai. 6. Hệ thống hiển thị số lượng Sinh viên bị 0 điểm vì không nộp, và khoá toàn bộ bảng điểm (không thể chấm/sửa thêm). 7. Sinh viên trong lớp có thể xem điểm và nhận xét của bài tập.
Alternative Flows	Alternative flow 1: Hủy xác nhận 1. Từ bước #3, Giảng viên nhấn “Hủy” ở hộp thoại xác nhận. 2. Hệ thống không thực hiện gì thêm; bảng điểm vẫn ở trạng thái có thể chỉnh sửa.
Pre-conditions	Giảng viên là chủ lớp chứa bài tập; bảng điểm của bài tập chưa từng được công bố.
Post-conditions	Điểm của bài tập bị khoá vĩnh viễn, hiển thị công khai cho Sinh viên trong lớp; không thể chấm/sửa lại qua giao diện.
Quan hệ include / extend	Không có quan hệ include/extend.

2.12 Use-case: Tạo bài tập tự luyện

Nhóm: Nghiệp vụ

Use case Name	Tạo bài tập tự luyện
Brief description	Use case mô tả cách Giảng viên tạo một bài tập luyện tập tự do, công khai cho mọi Sinh viên, không gán hạn nộp.
Actors	Giảng viên
Basic Flow	1. Giảng viên vào “Tạo bài tập tự do”, nhập Tiêu đề, chọn Độ khó (Dễ/Trung bình/Khó) và nhập Đề bài. 2. Giảng viên thêm các testcase mẫu (input/expected output).

	3. Giảng viên nhấn “Tạo bài tập”. 4. Hệ thống lưu bài tập ở trạng thái công khai cùng danh sách testcase tương ứng. 5. Bài tập xuất hiện trong danh sách bài luyện tập công khai cho mọi Sinh viên.
Alternative Flows	Alternative flow 1: Thiếu thông tin 1. Từ bước #3, thiếu Tiêu đề hoặc Đề bài. 2. Hệ thống báo lỗi “Thiếu thông tin”. Alternative flow 2: Chỉnh sửa bài tập tự luyện đã tạo 1. Giảng viên (chủ bài tập) sửa tiêu đề/độ khó/đề bài/testcase. 2. Hệ thống cập nhật bản ghi và thay toàn bộ testcase nếu có gửi lại danh sách mới.
Pre-conditions	Giảng viên đã đăng nhập.
Post-conditions	Bài tập tự luyện mới được công khai cho tất cả Sinh viên, không gắn với lớp học hay hạn nộp cụ thể.
Quan hệ include / extend	Không có quan hệ include/extend.

2.13 Use-case: Làm và nộp bài tự luyện

Nhóm: Nghiệp vụ

Use case Name	Làm và nộp bài tự luyện
Brief description	Use case mô tả cách Sinh viên tự luyện tập với một bài tập tự do và nhận kết quả chấm điểm ngay lập tức.
Actors	Sinh viên (chính); Gemini API, Groq API (phụ — dịch vụ AI bên ngoài)
Basic Flow	1. Sinh viên chọn một bài trong danh sách bài luyện tập công khai, chọn ngôn ngữ và viết code. 2. Sinh viên nhấn “Chạy thử” để xem kết quả thực tế trên các testcase mẫu (không lưu). 3. Sinh viên nhấn “Nộp bài”. 4. Hệ thống lưu bài nộp rồi ngay lập tức kích hoạt chấm tự động, dùng cùng pipeline testcase/độ phức tạp/phong cách/tổng kết như bài tập có hạn nộp, nhưng bỏ qua bước Gian lận vì đây là bài luyện tập cá nhân. 5. Hệ thống trả điểm, số testcase đạt/tổng số và nhận xét AI ngay cho Sinh viên xem.
Alternative Flows	Alternative flow 1: Chấm tự động lỗi hoặc mô hình chưa cấu hình 1. Từ bước #4, việc gọi chấm bị lỗi (ví dụ mô-đun chấm chưa cài hoặc quá thời gian chờ).

	2. Hệ thống vẫn giữ bản nộp ở trạng thái chưa chấm, không chặn việc nộp bài. Alternative flow 2: Nộp nhiều lần 1. Sinh viên nộp lại bài cùng một đề nhiều lần. 2. Hệ thống lưu mỗi lần nộp là một bản ghi riêng để giữ lịch sử, không ghi đè lần nộp trước.
Pre-conditions	Sinh viên đã đăng nhập; bài luyện tập đang ở trạng thái công khai.
Post-conditions	Một bản ghi nộp bài mới được lưu kèm điểm và nhận xét AI (nếu chấm thành công), phục vụ Sinh viên tự luyện tập và tự đánh giá.
Quan hệ include / extend	«include» Chấm bài nộp (kích hoạt ngay khi nộp; KHÔNG kèm Phát hiện gian lận). «extend» Chạy thử code (tùy chọn, trước khi nộp).

2.14 Use-case: Xuất bảng điểm ra Excel

Nhóm: Quản trị hệ thống — Quản trị dữ liệu

Use case Name	Xuất bảng điểm ra Excel
Brief description	Use case mô tả cách Giảng viên xuất toàn bộ bảng điểm của một lớp học ra file Excel để lưu trữ/báo cáo.
Actors	Giảng viên
Basic Flow	1. Giảng viên vào trang chi tiết lớp học, chọn tab “Bảng điểm”. 2. Hệ thống hiển thị bảng điểm tổng hợp: mỗi dòng là một Sinh viên, mỗi cột là một bài tập trong lớp, giá trị là điểm đã chấm (chỉ hiện điểm ở bài đã “graded”). 3. Giảng viên nhấn “Xuất Excel”. 4. Hệ thống dựng sheet “Bảng điểm” (cột STT, MSSV, Họ tên và một cột cho mỗi bài tập) từ dữ liệu bảng điểm hiện tại. 5. Hệ thống xuất và tải xuống file “bang-diem-<tên lớp>.xlsx” ngay trên trình duyệt.
Alternative Flows	Alternative flow 1: Lớp chưa có bài tập nào 1. Từ bước #3, lớp học chưa có bài tập nào được tạo. 2. Nút “Xuất Excel” bị vô hiệu hoá.
Pre-conditions	Giảng viên là chủ lớp học; lớp có ít nhất một bài tập.
Post-conditions	File Excel chứa bảng điểm hiện tại của lớp được tải xuống máy của Giảng viên; dữ liệu trên hệ thống không thay đổi.
Quan hệ include / extend	Không có quan hệ include/extend. Thuộc nhóm quản trị dữ liệu (kết xuất báo cáo), xử lý hoàn toàn phía trình duyệt bằng thư viện xlsx.

3. Đặc tả rút gọn các use case «include» / «extend»

Các use case dưới đây phát sinh từ quan hệ include/extend, được tách riêng để tái sử dụng và làm rõ luồng chấm điểm. Cột cuối chỉ tới vị trí hiện thực trong mã nguồn để tiện đối chiếu.

Use case	Loại	Mô tả & vị trí trong mã nguồn
Gửi email (mã/liên kết) <i>Được dùng bởi: Đăng ký tài khoản; Quên & đặt lại mật khẩu</i>	«include»	Sinh nội dung email (mã 6 số hoặc link) và gửi qua SMTP; ở môi trường DEV in link/mã ra console thay vì gửi thật. <i>backend/src/lib/email.ts, lib/tokens.ts</i>
Gửi lại mã xác nhận <i>Được dùng bởi: Xác nhận email</i>	«extend»	Khi người dùng chủ động yêu cầu, sinh verify_token mới và gửi lại; trả thông báo chung để chống dò email. <i>routes/auth.ts (resend-verification)</i>
Chạy thử code <i>Được dùng bởi: Nộp bài tập; Làm & nộp bài tự luyện</i>	«extend»	Chạy code với testcase công khai để xem kết quả tức thời, KHÔNG lưu và KHÔNG tính điểm. <i>grading/sandbox.ts, routes/submissions.ts (run)</i>
Nộp kèm file <i>Được dùng bởi: Nộp bài tập</i>	«extend»	Cho phép tải file code lên thay vì dán trực tiếp; lưu file vào thư mục upload và ghi đường dẫn vào bài nộp. <i>lib/upload.ts (multer), routes/submissions.ts</i>
Xem code bài nộp <i>Được dùng bởi: Chỉnh sửa điểm bài nộp</i>	«extend»	Mở toàn bộ code của một bài nộp để Giảng viên đối chiếu trước khi chỉnh điểm. <i>features/lecturer (bảng điểm/chi tiết bài nộp)</i>
Chấm bài nộp <i>Được dùng bởi: Chấm điểm tự động bài tập; Làm & nộp bài tự luyện</i>	«include»	Điều phối chấm 1 bài nộp: Testcase → (nếu chạy được) Độ phức tạp // Phong cách → Tổng hợp. Dùng chung cho cả hai luồng chấm. <i>grading/pipeline.ts → gradeSubmission()</i>
Phát hiện gian lận <i>Được dùng bởi: Chấm điểm tự động bài tập (GATE — chỉ ở chế độ assignment)</i>	«include»	So khớp k-gram/winnowing giữa các bài nộp; vượt ngưỡng rubric → 0 điểm và dừng pipeline cho bài đó. KHÔNG áp dụng cho bài tự luyện. <i>grading/plagiarism.ts</i>
Chạy testcase (sandbox) <i>Được dùng bởi: Chấm bài nộp</i>	«include»	Biên dịch/chạy code trong sandbox, chuẩn hoá và so output với expected → % testcase đạt. <i>grading/testcase.ts, grading/sandbox.ts</i>
Phân tích độ phức tạp <i>Được dùng bởi: Chấm bài nộp</i>	«include»	Phân tích tĩnh cấu trúc code, ước lượng lớp Big-O rồi map sang phần trăm theo các mốc rubric. <i>grading/complexity.ts</i>
Chấm phong cách code <i>Được dùng bởi: Chấm bài nộp</i>	«include»	Gọi Gemini API nhận xét phong cách (đặt tên/cấu trúc/độ rõ ràng); chỉ nhận xét, không cộng trọng số điểm. <i>grading/style.ts → Gemini (generativelanguage.googleapis.com)</i>

Use case	Loại	Mô tả & vị trí trong mã nguồn
Tổng hợp điểm & nhận xét <i>Được dùng bởi: Chấm bài nộp</i>	«include»	Tính điểm cuối tất định theo trọng số rubric rồi gọi Groq API viết nhận xét tổng hợp 4 mô-đun. <i>grading/aggregate.ts → Groq (api.groq.com)</i>
Gợi ý sửa lỗi (AI) <i>Được dùng bởi: Chấm bài nộp (khi code không biên dịch/chạy được)</i>	«extend»	Điểm mở rộng: khi testcase báo lỗi nghiêm trọng, chuyển module phong cách (Gemini) sang chế độ debug để chỉ ra lỗi và hướng dẫn sửa; KHÔNG tính điểm tổng kết. <i>grading/style.ts (purpose = 'debug'), pipeline.ts (nhánh tc.fatal)</i>